

Լարորատոր աշխատանք 5

Առաքման ծառայության պատվերներ

Աշխատանքի նպատակ. Ստեղծել ծրագիր, որը հնարավորություն է տալիս պահպանել և մշակել առաքման ծառայության պատվերների տվյալները, հաշվարկել առաքման արժեքը, գտնել ամենահեռու պատվերը, ցուցադրել չառաքված պատվերները և փոփոխել պատվերի առաքման կարգավիճակը:

Աշխատանքի պահանջը.

- Օգտագործել struct՝ առաքման պատվերի տվյալները պահելու համար:
- Մուտքագրել 12 առաքման պատվերի տվյալներ:
- Յուրաքանչյուր պատվերի համար պահպանել՝ կոդը, հասցեն, հեռավորությունը, քաշը և առաքման կարգավիճակը:
- Օգտագործել առանձին ֆունկցիաներ տվյալների մշակման համար:
- Հաշվել յուրաքանչյուր պատվերի առաքման արժեքը:
- Գտնել ամենահեռու առաքումը:
- Ցուցադրել չառաքված պատվերները:
- Տրված կոդով պատվերը նշել որպես առաքված:
- Հաշվել չառաքված պատվերների ընդհանուր արժեքը:
- Մուտքագրվող տվյալների համար կատարել սխալների ստուգում:

Առաջադրանք. Ստեղծել Delivery կառուցվածք՝ հետևյալ դաշտերով՝ code, address, distance, weight, delivered: Մուտքագրել 12 պատվեր և ֆունկցիաների միջոցով՝

1. Հաշվել յուրաքանչյուր պատվերի արժեքը՝ $\text{արժեք} = \text{distance} \times 100 + \text{weight} \times 50$
2. Գտնել ամենահեռու պատվերը:
3. Արտածել բոլոր չառաքված պատվերները:
4. Տրված code-ով պատվերը նշել որպես առաքված:
5. Հաշվել չառաքված պատվերների ընդհանուր արժեքը:

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

struct Delivery {
    int code;
    string address;
    double distance;
    double weight;
    bool delivered;
};

double calculateCost(const Delivery& d) {
    return d.distance * 100 + d.weight * 50;
}

int findFarthest(const Delivery deliveries[], int n) {
    int maxIndex = 0;

    for (int i = 1; i < n; i++) {
        if (deliveries[i].distance > deliveries[maxIndex].distance) {
            maxIndex = i;
        }
    }

    return maxIndex;
}

void printUndelivered(const Delivery deliveries[], int n) {
```

```
cout << "\n===== ՉԱՌԱՔՎԱԾ ՊԱՏՎԵՐՆԵՐ =====\n";
```

```
bool found = false;
```

```
for (int i = 0; i < n; i++) {  
    if (!deliveries[i].delivered) {  
        cout << "Code: " << deliveries[i].code << endl;  
        cout << "Address: " << deliveries[i].address << endl;  
        cout << "Distance: " << deliveries[i].distance << endl;  
        cout << "Weight: " << deliveries[i].weight << endl;  
        cout << "Cost: " << calculateCost(deliveries[i]) << endl;  
        cout << "-----\n";
```

```
        found = true;
```

```
    }
```

```
}
```

```
if (!found) {  
    cout << "Չկան չառաքված պատվերներ:\n";
```

```
}
```

```
}
```

```
void markAsDelivered(Delivery deliveries[], int n, int code) {  
    for (int i = 0; i < n; i++) {  
        if (deliveries[i].code == code) {  
            deliveries[i].delivered = true;  
            cout << "Պատվերը նշվեց որպես առաքված:\n";  
        }  
    }  
    return;
```

```
}  
}
```

```
cout << "Այդ code-ով պատվեր չկա:\n";
```

```
}
```

```
double totalUndeliveredCost(const Delivery deliveries[], int n) {
```

```
double total = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
if (!deliveries[i].delivered) {
```

```
total += calculateCost(deliveries[i]);
```

```
}
```

```
}
```

```
return total;
```

```
}
```

```
int main() {
```

```
const int N = 12;
```

```
Delivery deliveries[N];
```

```
for (int i = 0; i < N; i++) {
```

```
cout << "\n===== \n";
```

```
cout << "Ամսաթիվ #" << i + 1 << endl;
```

```
cout << "===== \n";
```

```
while (true) {
```

```
cout << "Code: ";

if (cin >> deliveries[i].code && deliveries[i].code > 0) {
break;
}

cout << "Սխալ է: Code-ը պետք է լինի դրական թիվ:\n";

cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
cout << "Address: ";
cin >> deliveries[i].address;

while (true) {
cout << "Distance: ";

if (cin >> deliveries[i].distance &&
deliveries[i].distance >= 0) {
break;
}

cout << "Սխալ է: Distance-ը պետք է լինի 0 կամ դրական թիվ:\n";

cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

```
while (true) {
    cout << "Weight: ";

    if (cin >> deliveries[i].weight &&
        deliveries[i].weight >= 0) {
        break;
    }

    cout << "Սխալ է: Weight-ը պետք է լինի 0 կամ դրական թիվ:\n";

    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

string answer;

while (true) {
    cout << "Delivered (yes/no): ";
    cin >> answer;

    if (answer == "yes") {
        deliveries[i].delivered = true;
        break;
    }

    if (answer == "no") {
        deliveries[i].delivered = false;
        break;
    }
}
```

```
}
```

```
cout << "Սխալ է: Պրեք միայն yes կամ no:\n";
```

```
}
```

```
}
```

```
cout << "\n===== ԱՌԱՔՈՒՄՆԵՐԻ ԱՐԺԵՔՆԵՐ =====\n";
```

```
for (int i = 0; i < N; i++) {
```

```
cout << "Code " << deliveries[i].code
```

```
<< " = "
```

```
<< calculateCost(deliveries[i])
```

```
<< endl;
```

```
}
```

```
int farthest = findFarthest(deliveries, N);
```

```
cout << "\n===== ԱՄԵՆԱՅԵՌՈՒ ԱՌԱՔՈՒՄ =====\n";
```

```
cout << "Code: "
```

```
<< deliveries[farthest].code << endl;
```

```
cout << "Address: "
```

```
<< deliveries[farthest].address << endl;
```

```
cout << "Distance: "
```

```
<< deliveries[farthest].distance << endl;
```

```
cout << "Weight: "
```

```
<< deliveries[farthest].weight << endl;
printUndelivered(deliveries, N);
int code;

while (true) {
    cout << "\nՈ՞ր պատվերը նշել որպես առաքված: ";

    if (cin >> code) {
        bool found = false;

        for (int i = 0; i < N; i++) {
            if (deliveries[i].code == code) {
                found = true;
                break;
            }
        }

        if (found) {
            markAsDelivered(deliveries, N, code);
            break;
        }

        cout << "Այդ code-ով պատվեր չկա: Փորձեք նորից:\n";
    }
    else {
        cout << "Սխալ է: Մուտքագրեք թիվ:\n";
    }
}
```

```

cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
}
cout << "\n===== ՉԱՌԱՔՎԱԾ ՊԱՏՎԵՐՆԵՐԻ ԸՆԴՀԱՆՈՒՐ ԱՐԺԵՔ =====\n";

cout << totalUndeliveredCost(deliveries, N) << endl;

return 0;
}

```

Օրինակային մուտք.

Առաքում #1

Code: 101

Address: Yerevan

Distance: 10

Weight: 2

Delivered (yes/no): yes

Առաքում #2

Code: 102

Address: Gyumri

Distance: 120

Weight: 5

Delivered (yes/no): no

Առաքում #3

Code: 103

Address: Vanadzor

Distance: 115

Weight: 3

Delivered (yes/no): yes

Unupnuł #4

Code: 104

Address: Dilijan

Distance: 100

Weight: 4

Delivered (yes/no): no

Unupnuł #5

Code: 105

Address: Sevan

Distance: 70

Weight: 2

Delivered (yes/no): no

Unupnuł #6

Code: 106

Address: Hrazdan

Distance: 50

Weight: 6

Delivered (yes/no): yes

Անսլոյ #7

Code: 107

Address: Goris

Distance: 250

Weight: 4

Delivered (yes/no): no

Անսլոյ #8

Code: 108

Address: Kapan

Distance: 300

Weight: 7

Delivered (yes/no): no

Անսլոյ #9

Code: 109

Address: Artashat

Distance: 30

Weight: 3

Delivered (yes/no): yes

Անսլոյ #10

Code: 110

Address: Armavir

Distance: 45

Weight: 5

Delivered (yes/no): no

Առաքում #11

Code: 111

Address: Echmiadzin

Distance: 25

Weight: 2

Delivered (yes/no): yes

Առաքում #12

Code: 112

Address: Jermuk

Distance: 175

Weight: 4

Delivered (yes/no): no

Ո՞ր պատվերը նշել որպես առաքված: 102

Օրինակային ելք .

===== ԱՌԱՔՈՒՄՆԵՐԻ ԱՐԺԵՔՆԵՐ =====

Code 101 = 1100

Code 102 = 12250

Code 103 = 11650

Code 104 = 10200

Code 105 = 7100

Code 106 = 5300

Code 107 = 25200

Code 108 = 30350

Code 109 = 3150
Code 110 = 4750
Code 111 = 2600
Code 112 = 17700

===== ԱՄԵՆԱՅԵՌՈՒ ԱՌԱՔՈՒՄ =====

Code: 108
Address: Kapan
Distance: 300
Weight: 7

===== ՉԱՌԱՔՎԱԾ ՊԱՏՎԵՐՆԵՐ =====

Code: 102
Address: Gyumri
Distance: 120
Weight: 5
Cost: 12250

Code: 104
Address: Dilijan
Distance: 100
Weight: 4
Cost: 10200

Code: 105

Address: Sevan

Distance: 70

Weight: 2

Cost: 7100

Code: 107

Address: Goris

Distance: 250

Weight: 4

Cost: 25200

Code: 108

Address: Kapan

Distance: 300

Weight: 7

Cost: 30350

Code: 110

Address: Armavir

Distance: 45

Weight: 5

Cost: 4750

Code: 112

Address: Jermuk

Distance: 175

Weight: 4

Cost: 17700

Ո՞ր պատվերը նշել որպես առաքված: 102

Պատվերը նշվեց որպես առաքված:

===== ՉԱՌԱՔՎԱԾ ՊԱՏՎԵՐՆԵՐԻ ԸՆԴՀԱՆՈՒՐ ԱՐԺԵՔ =====

85300